LifeSaver — AI-Powered Deadline Companion

Google AI Studio Hackathon 2026 | Submission Document

Problem Statement Selected

The Last-Minute Life Saver

> *"Build an AI-powered productivity companion that proactively assists users in planning, prioritizing, and actually completing tasks before they miss deadlines."*

Deadline anxiety is one of the most universal and damaging productivity problems. Students miss submissions. Professionals delay deliverables. Developers push broken code at 2 AM. The core failure isn't laziness — it's the absence of intelligent, personalized planning support that acts *before* the crisis hits.

Existing tools (Todoist, Notion, Google Tasks) are passive — they remind you something is due, but they don't help you figure out *when to start*, *how to break it down*, or *what to do when you fall behind*. They treat every user identically, ignoring the fact that you consistently underestimate ML tasks by 3 hours or always finish writing tasks faster than planned.

LifeSaver solves this by replacing passive reminders with an autonomous AI agent system that plans, schedules, monitors, and re-plans — without being asked.

Solution Overview

LifeSaver is a multi-agent AI productivity system built on Google AI Studio. A user describes a task in plain English — *"Submit ML assignment before Friday, it's complex and I haven't started"* — and five specialized AI agents immediately take over:

Agent 1 (Parser) converts the natural language input into a structured task object, extracting deadline, complexity, category, and estimated effort.

Agent 2 (Prioritization) uses a RAG (Retrieval-Augmented Generation) pipeline to search the user's personal task history, retrieving semantically similar past tasks and surfacing behavioral insights — *"Last time you had a high-complexity academic task, you underestimated by 3 hours"* — to compute personalized urgency, importance, and risk scores.

Agent 3 (Planning) decomposes the task into 3–8 concrete, actionable subtasks with realistic time estimates, execution tips, and a logical ordering based on dependencies.

Agent 4 (Scheduler) reads the user's Google Calendar freebusy data, computes available time slots between now and the deadline, and automatically books calendar events for each subtask — fitting them into real gaps in the user's schedule.

Agent 5 (Monitor) runs autonomously every 30 minutes via a cron job. It recomputes the live risk score for every active task, detects when subtasks are overdue, escalates when deadlines are critically close, and triggers autonomous re-planning — rescheduling remaining subtasks in the calendar without any user action.

Every step of this pipeline streams to the user interface in real-time via Server-Sent Events, showing exactly what each agent is thinking as it works — making the AI reasoning visible and trustworthy.

Key Features

1. Natural Language Task Input

Users describe tasks exactly as they would to a colleague. No forms, no dropdowns, no forced structure. The Parser Agent handles ambiguity, infers missing information, and asks for clarification only when truly necessary.

2. Live Agent Trace Panel

A terminal-style panel streams each agent's thoughts in real-time as they process the task. Users see the system working — *"Searching your task history... Found 3 relevant past tasks... Computing risk score..."* — making the AI pipeline transparent and impressive to interact with.

3. RAG-Powered Personalization

Every completed task is embedded using Gemini's `embedding-001` model and stored in a Firestore-backed vector store. When a new task arrives, semantic search retrieves the most relevant past experiences. The Prioritization Agent uses this context to adjust estimates and risk scores based on *that specific user's* behavioral patterns — not generic averages.

4. Autonomous Risk Scoring

Each task carries a live 0–100 risk score displayed as an animated arc gauge. The Monitor Agent recomputes this every 30 minutes using a formula that accounts for time elapsed vs. subtasks completed, overdue scheduled blocks, and hours remaining until deadline. The score updates without any user action.

5. Automatic Calendar Scheduling

Subtasks are automatically booked into the user's Google Calendar using the freebusy API to find real available slots. Events appear in the calendar with titles, descriptions, tips, and reminder popups — 10 minutes before and 1 minute before each block.

6. Autonomous Re-planning

When the Monitor Agent detects a risk score above 80 and more than 1 hour remaining, it automatically deletes outdated calendar events and reschedules the remaining subtasks into new available slots — without the user doing anything. This is the defining autonomous behaviour of the system.

7. Deadline Escalation

When fewer than 2 hours remain and a task is incomplete, the Monitor Agent sets an escalation flag and generates an urgent warning message shown prominently in the UI.

8. Timeline View

A visual timeline groups all scheduled subtasks across all active tasks by day, showing the user exactly what they need to work on and when — color-coded by risk level.

9. Task History & Outcome Recording

When a user marks a task complete, the actual hours taken and minutes late are recorded and embedded back into the RAG store — feeding future personalization. The system genuinely learns from each completed task.

Technologies Used

Backend

Technology	Version	Purpose
Node.js	v22	Server runtime
Express.js	v4.18	HTTP server and API routing
node-cron	v3.0	Cron scheduler for Monitor Agent
uuid	v9.0	Unique IDs for tasks and subtasks
express-session	v1.17	Session management for OAuth flow
dotenv	v16	Environment variable management

Frontend

Technology	Version	Purpose
React	v18.2	UI framework
Vite	v5.0	Build tool and dev server
Tailwind CSS	v3.3	Utility-first styling
React Router	v6.20	Client-side routing
date-fns	v3.0	Date formatting and calculations
EventSource API	Native	Server-Sent Events for live agent trace

Architecture Patterns

- **Multi-Agent Pipeline** — 5 specialized agents chained via an orchestrator
- **RAG (Retrieval-Augmented Generation)** — Cosine similarity search over user-specific embeddings
- **Server-Sent Events (SSE)** — Real-time streaming of agent trace to the frontend
- **OAuth 2.0** — Google Calendar authorization with offline refresh token support

---

Google Technologies Utilized

1. Google AI Studio — Gemini 2.5 Flash

Primary AI engine powering all 5 agents.

Used for:

- **Agent 1 (Parser):** Structured extraction of task metadata from natural language
- **Agent 2 (Prioritization):** Multi-factor scoring with RAG context injection
- **Agent 3 (Planning):** Subtask decomposition with time estimation and tips
- **Agent 5 (Monitor):** Risk assessment language generation and escalation messages

Configuration: `temperature: 0.3` for planning tasks (consistency), `temperature: 0.1` for parsing (determinism), `maxOutputTokens: 4096` for complex plans.

All agents prompt Gemini to respond in strict JSON format, with a custom `parseGeminiJSON()` utility that strips markdown fences before parsing.

2. Google AI Studio — Gemini Embedding Model (`embedding-001`)

Powers the RAG personalization engine.

Every task — when created and when completed — is converted to a 768-dimensional vector embedding. These embeddings are stored in Firestore alongside task metadata. When a new task arrives, its embedding is compared to all stored embeddings using cosine similarity, and the top 5 most semantically relevant past experiences are retrieved and injected into the Prioritization Agent's prompt.

This means a user who always underestimates coding tasks will see that pattern reflected in their risk scores from the very first recurrence.

3. Firebase Authentication

Google Sign-In for instant, frictionless onboarding.

Users authenticate with a single click using their Google account. Firebase Authentication issues ID tokens that are verified server-side using the Firebase Admin SDK on every protected API request. This ensures complete data isolation — each user only ever sees their own tasks and history.

4. Firebase Firestore

Primary database for tasks, user profiles, RAG vector store, and calendar tokens.

Collections:

- `tasks/{taskId}` — Full task document including all agent outputs, subtasks, scheduled slots, risk scores, and status
- `users/{userId}` — User profile with calendar connection status
- `users/{userId}/settings/calendar_tokens` — OAuth tokens for Google Calendar
- `users/{userId}/rag_entries/{entryId}` — Personal vector store entries with embeddings

Firestore's real-time capabilities and compound query support make it ideal for the Monitor Agent's batch sweeps across all active tasks.

5. Firebase Hosting

Frontend deployment platform.

The React/Vite application is built and deployed to Firebase Hosting, providing a global CDN, automatic HTTPS, and SPA routing support via rewrite rules. The deployed app is available at the submission URL and will remain active throughout the evaluation period.

6. Google Calendar API v3

Full read/write calendar integration for autonomous scheduling.

Used for:

- `freebusy.query` — Fetching busy time blocks to compute available slots
- `events.insert` — Creating calendar events for each scheduled subtask, with color coding (tangerine/orange for visibility), descriptions, and dual reminders (10 min + 1 min before)
- `events.delete` — Removing outdated events when the Monitor Agent triggers a re-plan

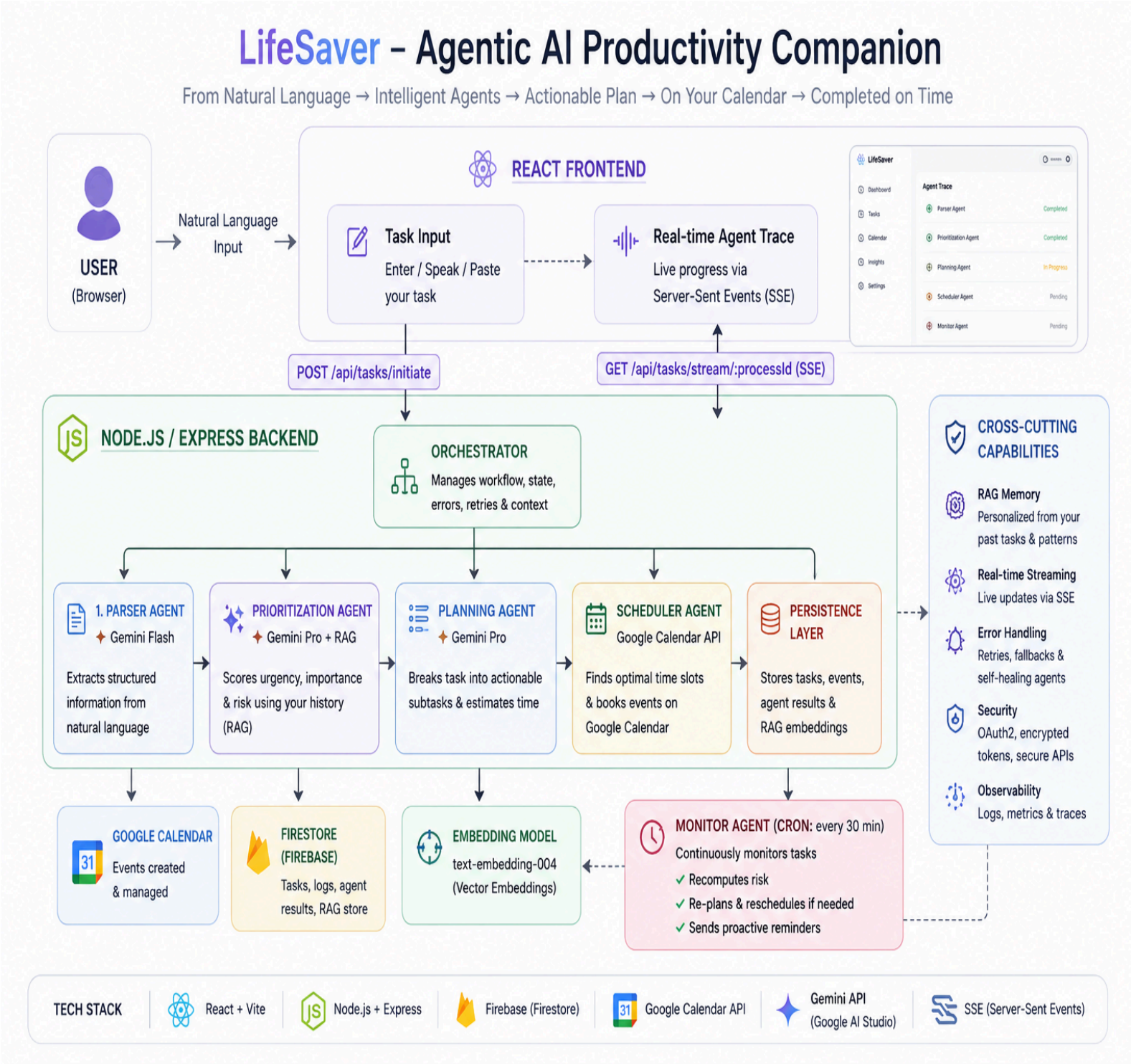
OAuth 2.0 with `offline` access type ensures refresh tokens are stored and reused, so users only authorize once and the system can schedule autonomously on their behalf in subsequent sessions.

7. Google Cloud Console

Infrastructure and credentials management.

- OAuth 2.0 credentials created and managed here for Calendar access
- Google Calendar API enabled via the API Library
- OAuth consent screen configured with required scopes

Architecture Diagram



Agentic Depth Explanation

The system demonstrates genuine agentic depth across three dimensions:

Specialization — Each agent has a single, well-defined responsibility and uses the model configuration best suited to it. The Parser uses low temperature for determinism. The Prioritization and Planning agents use higher token limits for richer reasoning. No single monolithic prompt handles everything.

Autonomy — Agent 5 (Monitor) runs on a cron schedule completely independent of user interaction. It reads the state of every active task, makes decisions about replanning, modifies the database, and updates the calendar — all without being triggered by the user. This is genuine autonomous behaviour, not a simulated one.

Learning — The RAG pipeline means agent behaviour changes over time for each user. A user who completes their first task provides data. On their second similar task, the Prioritization Agent retrieves that history and produces a different (more accurate) risk score. The system improves with use.

Team

Shailesh Pratap Singh
M.Tech - IT (IIIT-A)
AI/ML Researcher & Full-Stack Developer